

Plateforme TP C, vue d'ensemble

Application de bureau pour apprendre le langage C par la pratique, développée dans le cadre d'un stage 1A (ENSEEIH), sur le thème « intégrer l'IA générative dans l'apprentissage de la programmation ». L'étudiant fait les exercices du BE C ; l'appli compile son code et valide chaque exercice par une porte ; un tuteur IA aide sans donner la solution.

Ce que fait la plateforme

- **Un atelier PyQt6** : énoncé, éditeur de code C, console, tuteur, liste de progression.
- **Des portes qui compilent vraiment** : le code de l'étudiant est compilé avec `gcc` et exécuté ; la porte s'ouvre si la sortie correspond à l'attendu.
- **Un tuteur IA bridé** : appelle `claude` en ligne de commande, avec un garde-fou structurel qui rejoue la porte sur la réponse pour retirer toute solution.
- **Un mode auteur** : ajouter, modifier, réordonner et retirer des niveaux depuis l'appli, protégé par mot de passe.
- **Une remontée Moodle optionnelle** : un compagnon LTI renvoie les scores au carnet de notes.

Principe : le contenu est une donnée

Un parcours n'est pas codé en dur, c'est un dossier sous `contenu\`. On passe d'un parcours à l'autre sans toucher au code. Le parcours de référence est `be_c`, les 14 exercices du BE C dans l'ordre des slides. Voir le guide auteur pour le format.

Lancer

Double-clic sur `Atelier.bat`. Le lanceur détecte Python, ajoute le compilateur portable `w64devkit\bin` au PATH, et ouvre le dernier parcours retenu.

En ligne de commande :

<code>python atelier_snake.py</code>	dernier parcours retenu (default <code>be_c</code>)
<code>python atelier_snake.py --parcours be_c</code>	parcours precis
<code>python atelier_snake.py --demo</code>	mode demo, tout debloque
<code>python atelier_snake.py --selftest</code>	verifie les portes sans ecran
<code>python atelier_snake.py --smoketest</code>	construit la fenetre sans l'afficher

Structure du dépôt

Élément	Contenu
<code>atelier_snake.py</code>	Point d'entrée, choix du parcours, modes de test
<code>fenetre.py</code>	La fenêtre principale (UI)
<code>executeur.py</code>	Compilation, exécution, portes
<code>modele_etape.py</code>	Lecture des niveaux et parcours
<code>tuteur_ia.py</code>	Le tuteur IA (multi-moteur claude / codex)
<code>garde_fous.py</code>	Garde-fou anti-solution du tuteur
<code>progression.py</code>	État de progression, crans d'aide
<code>coloration.py</code>	Coloration syntaxique C
<code>lsp_clangd.py</code>	Diagnostics live via clangd (optionnel)
<code>moodle_sync.py</code>	Remontée des scores vers le compagnon Moodle
<code>gestion_niveaux.py</code>	Logique du mode auteur (ajout, retrait, ordre, édition)
<code>auteur.py</code>	Mot de passe du mode auteur
<code>reglages.py</code>	Dernier parcours retenu
<code>diagnostic.py</code>	Chemins clés et présence des outils
<code>dialogue_*.py</code>	Les fenêtres du mode auteur et du diagnostic
<code>contenu\</code>	Les parcours (dont <code>be_c</code>)
<code>compagnon\</code>	Le service LTI Moodle (Flask), déployé séparément
<code>packaging\</code>	Fichiers du bundle portable Windows
<code>tests\</code>	Tests unittest
<code>docs\</code>	Cette documentation

Outils requis

Outil	Rôle	Obligatoire
Python	Fait tourner l'appli (avec PyQt6)	Oui
<code>gcc</code>	Compile le C des exercices	Oui pour les portes
<code>clangd</code>	Diagnostics live dans l'éditeur	Non
<code>claude</code>	Tuteur IA	Non

Le bundle portable embarque Python, PyQt6, `gcc` (w64devkit) et `clangd`. L'appli dégrade proprement quand `clangd` ou `claude` manquent.

Tests

```
python -m unittest discover -s tests
```

Il faut `gcc` au PATH pour les tests qui compilent (ce que fait `Atelier.bat`). Les tests du parcours projet (Snake) exigent SDL3 et sont hors périmètre sous Windows.

Le compagnon Moodle

`compagnon\` est un service Flask séparé (LTI 1.3), déployé sur un hébergeur type Render. Il gère l'appairage par code court et renvoie les scores au carnet Moodle via AGS. Voir `compagnon\README.md`. La plateforme marche sans : la connexion Moodle est optionnelle.

Documentation

- `docs\01-guide-utilisateur.md` : pour l'étudiant ou le 2A qui reçoit la plateforme.
- `docs\02-guide-auteur.md` : créer et modifier le contenu.
- `docs\03-README-projet.md` : ce document.
- `docs\04-doc-technique.md` : architecture pour reprendre le développement.